Лабораторная работа 3: Метод Гаусса

План реализаций для бенчмаркинга

1. Входные данные и параметры

Для всех версий алгоритма используются одинаковые входные данные:

1. **Размер матрицы:** $N = 2000$.
2. **Генерация матрицы A:** Случайные числа типа `double` в диапазоне от -100 до 100.
3. **Точное решение:** Вектор $x_{\text{true}} = (m, m+1, ..., N+m-1)$, где m — номер студента в группе.
4. **Правая часть (b):** Вычисляется как $b = A \cdot x_{\text{true}}$. Это позволяет нам знать истинный ответ и точно измерить погрешность.

2. Метрики для замера

Для каждого запуска программы в отчет должны попасть три ключевых показателя:

1. **Время выполнения:** В секундах (с помощью `chrono`).
2. **Норма невязки:** $\|b - Ax^*\|_2$ (показывает, насколько найденный x^* удовлетворяет уравнению).
3. **Относительная погрешность:** $\|x_{\text{true}} - x^*\|_2 / \|x_{\text{true}}\|_2$ (показывает, насколько найденный вектор отличается от точного). Также вывести первые 5 координат x^*.

3. Варианты реализаций (что нужно бенчмаркать)

3.1. Версия 1: Последовательный метод Гаусса (без выбора ведущего элемента)

Базовая реализация. Прямой ход идет сверху вниз, делит на текущий диагональный элемент $A[i][i]$.

1. **Цель:** Показать, что на матрицах 2000×2000 случайных чисел этот метод выдает огромную погрешность из-за деления на близкие к нулю элементы (или ловит `NaN`, если на диагонали окажется точный 0).

3.2. Версия 2: Последовательный метод Гаусса (с выбором ведущего элемента по столбцу)

Перед каждым шагом прямого хода ищем максимальный по модулю элемент в текущем столбце (ниже диагонали), меняем строки местами (вместе с `b`), и только потом исключаем переменные.

1. **Цель:** Доказать, что выбор главного элемента улучшает устойчивость. Ожидается, что время работы будет почти таким же, как в Версии 1, но относительная погрешность упадет на несколько порядков.

3.3. Версия 3: Параллельный метод Гаусса (без выбора ведущего элемента) [OpenMP]

Используем последовательный алгоритм из Версии 1, но распараллеливаем внутренние вычисления.

1. **Как параллелить:** Внешний цикл по столбцам `i` остается последовательным (шаги алгоритма зависят друг от друга). Внутренний цикл вычитания строк `j` распараллеливается через `#pragma omp parallel for`.
2. **Замеры:** Провести замеры для $p = 1, 2, 4, 8$ потоков. Оценить масштабируемость (speedup).

3.4. Версия 4: Параллельный метод Гаусса (с выбором ведущего элемента) [OpenMP]

Алгоритм из Версии 2 с распараллеленным внутренним циклом вычитания строк.

1. **Цель:** Показать, что добавление OpenMP не ломает математическую устойчивость (погрешность остается низкой), а время выполнения падает пропорционально числу потоков.

4. Дополнительные требования к бенчмаркингу (OpenMP)

1. **Избегание "дребезга" потоков:** Нельзя создавать параллельную область внутри цикла по `j` или `i`. Параллельная область (`#pragma omp parallel`) должна создаваться один раз на весь прямой ход, а внутри использоваться `#pragma omp for`.
2. **Память:** Матрица A и вектор b должны быть реализованы как плоские векторы (`vector<double>`) размером $N \times N$ и N соответственно, чтобы избежать проблем с кэшем при $N = 2000$.
3. **Сложность:** Прямой ход имеет сложность $O(N^3)$, поэтому при $N = 2000$ последовательные версии будут работать несколько секунд, а параллельные — значительно быстрее.